



**Islington college**  
(इस्लिङ्टन कलेज)

**Module Code & Module Title**

**CU6051NI Artificial Intelligence**

**75% Individual Coursework**

**Submission: Milestone 1**

**Academic Semester: Autumn Semester 2025**

**Credit: 15 credit semester long module**

**Student Name: MD Samim Rain**

**London Met ID: 23050299**

**College ID: np02cp4a230163**

**Assignment Due Date: 21/01/2026**

**Assignment Submission Date: 21/01/2026**

**Submitted To: Mr. Binod Bhattarai**

*I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.*

## Table of Contents

|                                                    |    |
|----------------------------------------------------|----|
| 1. Introduction .....                              | 5  |
| 1.1. Explanation of Topic .....                    | 5  |
| 1.2. Problem Domain .....                          | 5  |
| 2. Background .....                                | 6  |
| 3. Solution .....                                  | 6  |
| 3.1. Linear Regression .....                       | 6  |
| 3.2. Random Forest .....                           | 7  |
| 3.3. Ensemble Model .....                          | 7  |
| 3.4. Pseudocode of Linear Regression .....         | 7  |
| 3.5. Pseudocode of Random Forest .....             | 8  |
| 3.6. Pseudocode of Ensemble Model .....            | 9  |
| 3.7. Flowchart of Linear Regression .....          | 10 |
| 3.8. Flowchart of Random Forest .....              | 11 |
| 3.9. Flowchart of Ensemble Model .....             | 12 |
| 4. Development process .....                       | 13 |
| 4.1. Explanation of the Development Process .....  | 13 |
| 4.2. Tools, Technologies, and Libraries Used ..... | 13 |
| 4.3. Explanation of Development .....              | 14 |
| 4.4. Findings .....                                | 20 |
| 4.4.2. Random Forest .....                         | 22 |
| 5. Conclusion .....                                | 28 |
| 5.1. Analysis of the work done .....               | 28 |
| 5.2. Solution addresses real world problems .....  | 28 |
| 5.3. Future work .....                             | 28 |

|                       |    |
|-----------------------|----|
| 6. Bibliography ..... | 29 |
|-----------------------|----|

## Tables of Figures

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| Figure 1: Flowchart of Linear regression .....                               | 10 |
| Figure 2: flowchart of Random Forest .....                                   | 11 |
| Figure 3: Flowchart of ensemble model .....                                  | 12 |
| Figure 4: Importing dataset and libraries. ....                              | 14 |
| Figure 5: Checking missing values.....                                       | 14 |
| Figure 6: checking missing values aging after cleaning. ....                 | 15 |
| Figure 7: Checking datatype. ....                                            | 15 |
| Figure 9: Screenshot of statistical summary.....                             | 16 |
| Figure 10: Displaying dataset.....                                           | 16 |
| Figure 11: screenshot of bar chart. ....                                     | 17 |
| Figure 12: Figure of removing state missing and X. ....                      | 17 |
| Figure 13: showing states. ....                                              | 17 |
| Figure 14: Top ten calculation by spending drugs by Bar chart.....           | 18 |
| Figure 15: Screenshot of Pie chart.....                                      | 18 |
| Figure 16: Drug spending over time by state.....                             | 19 |
| Figure 17: Correlation of heatmap of drug spending and prescription.....     | 19 |
| Figure 18: screenshot of linear regression with model performance .....      | 20 |
| Figure 19: predicated heatmap in linear regression. ....                     | 20 |
| Figure 20: Total Spending and number of prescription.....                    | 21 |
| Figure 21: Screenshot of Random Forest model.....                            | 22 |
| Figure 22: Random Forest model performance.....                              | 23 |
| Figure 23: Random Forest of total spending and number of prescriptions. .... | 24 |
| Figure 24: Screenshot of ensemble model with voting Regressor. ....          | 25 |
| Figure 25: Train ensemble model with voting.....                             | 26 |
| Figure 26: Ensemble model performance. ....                                  | 27 |
| Figure 27: Total spending of ensemble model. ....                            | 27 |

## 1. Introduction

### 1.1. Explanation of Topic

Medicine in increasing many countries especially United State of America. Governments and health organizations spend a lot of money on medicines for public health programs. Because of this, it is important to understand how drug spending changes and what factors affect it. By studying drug spending data, I can find useful patterns and trends that help in better planning and decision-making.

In this project, a pharmaceutical drug spending dataset is used to analyses and predict drug costs. The dataset is taken from public sources provided by the U.S. government and is available on the Hugging Face platform (Utilization, n.d.). This dataset includes information such as state-wise drug utilization, total spending, and number of prescriptions. Machine learning is helpful in this case because it can learn from past data and make predictions about future drug spending (Datacatlog, 2025).

In this coursework I used three model which I predict the pharmaceutical drug spending using ML. Therefore, Linear Regression, Random Forest, and Ensemble Model I send which is aimed to predict the about medicine and compassion. When I compare these three models then I confirm which model is more accurately given the output best result for medicines predictions. Now this model shows how artificial intelligence technology can be applied to the real-world data predicted.

### 1.2. Problem Domain

The cost of medicines is very high in many countries, especially in the United States. Governments and hospitals spend a large amount of money every year to provide medicines to citizens. It is often difficult to know in advance how much money will be needed for future drug supplies (CMS.gov, n.d.).

Studying state-wise drug utilization data helps to understand which medicines are used more and which states spend more money. By analysing this data, patterns and trends in drug spending can be identified. This information is useful for planning budgets and making better decisions about medicine supply.

Machine learning can help predict future drug spending by learning from past data. Using models like Linear Regression, Random Forest, and Ensemble models, governments and hospitals can estimate future costs more accurately. This helps save money, plan budgets efficiently, and ensure that medicines are available for all people who need them.

## **2. Background**

Drug spending is increasing every year, and it is becoming a big challenge for governments and hospitals. Understanding how much money is spent on medicines and which states spend more is important for planning and budgeting. Many datasets about drug use and costs are now available from government sources, which can be used to analyze and predict spending.

In the first coursework, we studied the pharmaceutical drug spending dataset from the U.S. government, which includes state-wise drug utilization, total spending, and number of prescriptions. Using this data, we applied basic data cleaning, handled missing values, and visualized trends such as total spending per state and number of prescriptions (Medicaid.gov, n.d.).

Machine learning models like Linear Regression, Random Forest, and Ensemble models are helpful in predicting future spending because they can learn from past data. Previous studies show that Random Forest often gives better predictions than Linear Regression for healthcare cost data. Ensemble models, which combine multiple models, can improve accuracy and reduce errors (CMS.gov, n.d.).

## **3. Solution**

In this project, the main goal is to predict the total spending on pharmaceutical drugs in different U.S. states using machine learning. To do this, we used three different AI algorithms: Linear Regression, Random Forest, and an Ensemble Model.

### **3.1. Linear Regression**

Linear Regression is a simple machine learning model that predicts a numeric value based on the relationship between input features and the target value. In our case, the

input features are State, Drug Name, Number of Prescriptions, Drug Type, and the target is Total Spending. Linear Regression tries to draw a straight line that best fits the data points and predicts total spending for new data.

### 3.2. Random Forest

Linear Regression is a simple machine learning model that predicts a numeric value based on the relationship between input features and the target value. In our case, the input features are State, Drug Name, Number of Prescriptions, Drug Type, and the target is Total Spending. Linear Regression tries to draw a straight line that best fits the data points and predicts total spending for new data.

### 3.3. Ensemble Model

The Ensemble Model combines predictions from both Linear Regression and Random Forest to produce a better overall prediction. By combining the strengths of multiple models, the Ensemble Model can reduce errors and improve accuracy compared to using a single model.

### 3.4. Pseudocode of Linear Regression

#### **START**

IMPORT required Libraries (pandas, numpy, sklearn)

LOAD dataset (dataset.csv)

DESCRIBE dataset

CHECK null values

CLEAN data (fill numeric missing values with 0, categorical with 'Unknown')

DESCRIBE cleaned data

EXPLORE data (basic statistics, visualizations)

SELECT input features (State, Drug Name, Number of Prescriptions, Drug Type)

SELECT target variable (Total Spending)

ENCODE categorical features  
SPLIT data into training and testing set  
TRAIN Linear Regression model  
PREDICT Total Spending on testing set  
EVALUATE model performance

**END**

### 3.5. Pseudocode of Random Forest

**START**

IMPORT required Libraries (pandas, numpy, sklearn)  
LOAD dataset (dataset.csv)  
DESCRIBE dataset  
CHECK null values  
CLEAN data (fill numeric missing values with 0, categorical with 'Unknown')  
DESCRIBE cleaned data  
EXPLORE data (basic statistics, visualizations)  
SELECT input features (State, Drug Name, Number of Prescriptions, Drug Type)  
SELECT target variable (Total Spending)  
ENCODE categorical features  
SPLIT data into training and testing set  
TRAIN Random Forest model  
PREDICT Total Spending on testing set  
EVALUATE model performance (MAE, MSE,  $R^2$ )

**END**

### 3.6. Pseudocode of Ensemble Model

**START**

IMPORT required Libraries (pandas, numpy, sklearn)

LOAD dataset (dataset.csv)

DESCRIBE dataset

CHECK null values

CLEAN data (fill numeric missing values with 0, categorical with 'Unknown')

DESCRIBE cleaned data

EXPLORE data (basic statistics, visualizations)

SELECT input features (State, Drug Name, Number of Prescriptions, Drug Type)

SELECT target variable (Total Spending)

ENCODE categorical features

SPLIT data into training and testing set

TRAIN Linear Regression model

TRAIN Random Forest model

PREDICT Total Spending using Linear Regression on testing set

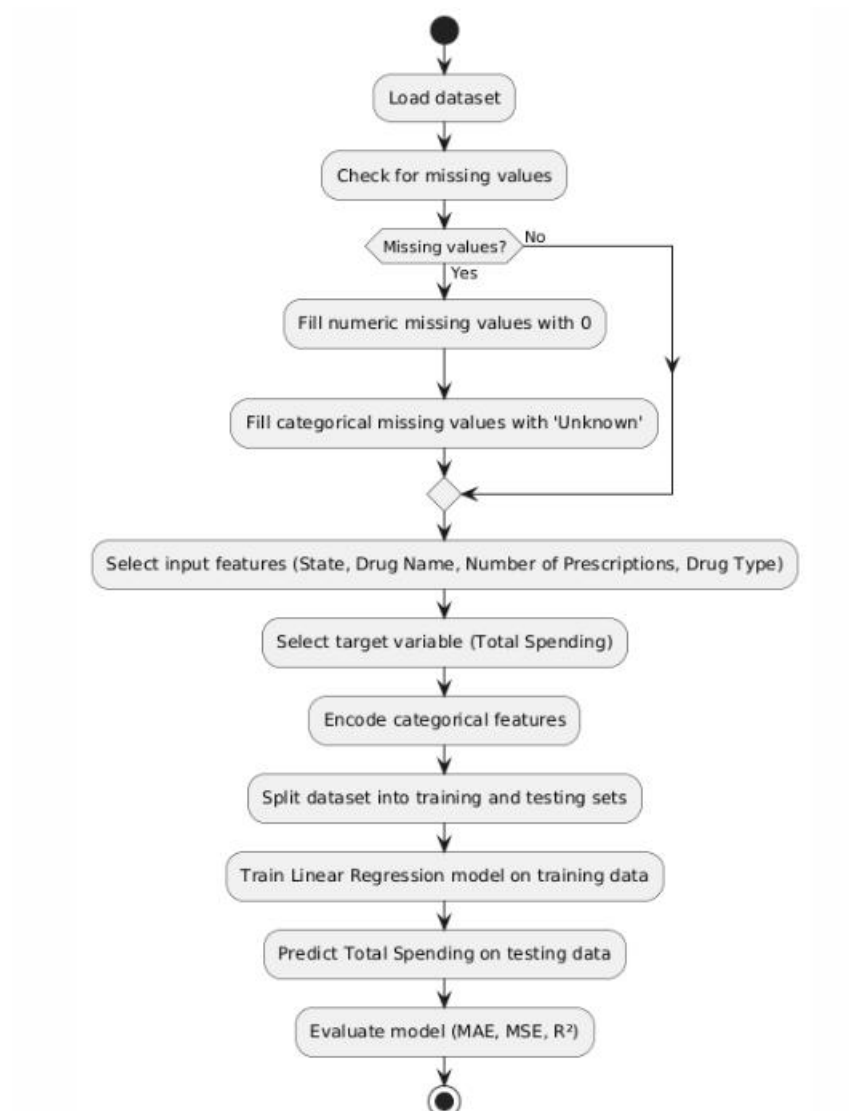
PREDICT Total Spending using Random Forest on testing set

COMBINE predictions from both models

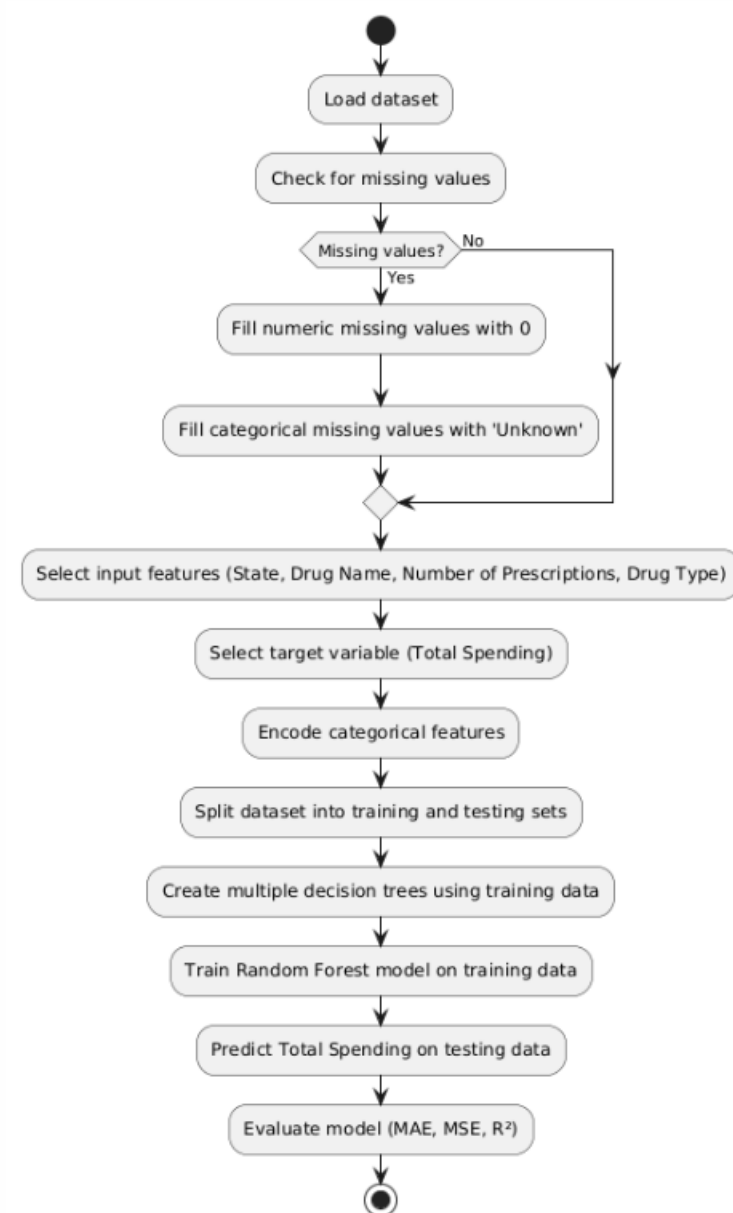
EVALUATE final Ensemble model performance (MAE, MSE,  $R^2$ )

**END**

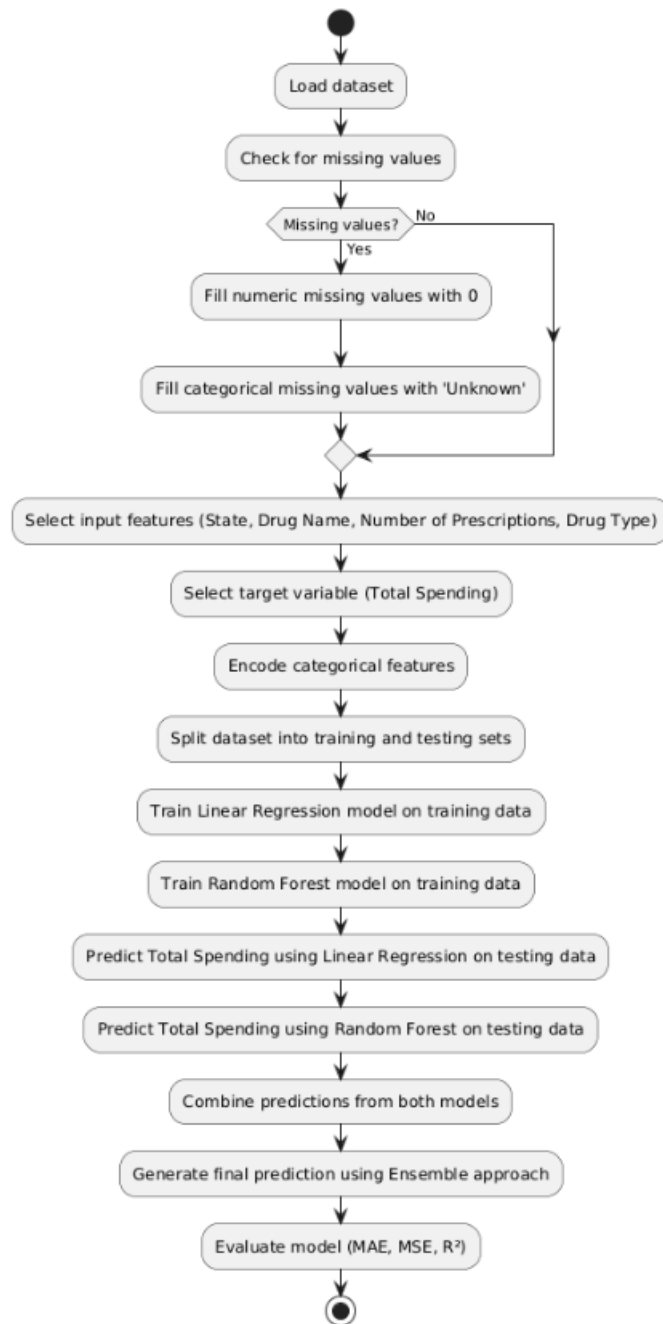
## 3.7. Flowchart of Linear Regression

*Figure 1: Flowchart of Linear regression*

## 3.8. Flowchart of Random Forest

*Figure 2: flowchart of Random Forest*

## 3.9. Flowchart of Ensemble Model

*Figure 3: Flowchart of ensemble model*

## 4. Development process

### 4.1. Explanation of the Development Process

In this project, the development process started by selecting a suitable dataset related to pharmaceutical drug spending. The dataset was collected from the Hugging Face platform, which provides public datasets from U.S. government sources. Python was chosen as the main programming language because it is easy to use and widely supported for data analysis and machine learning tasks.

Next, the missing values were checked and cleaned. Numeric missing values were filled with zero, and categorical missing values were filled with Unknown. This step was necessary to ensure that the dataset is complete and ready for machine learning models. After cleaning, the dataset was reviewed again to confirm that there were no missing values remaining.

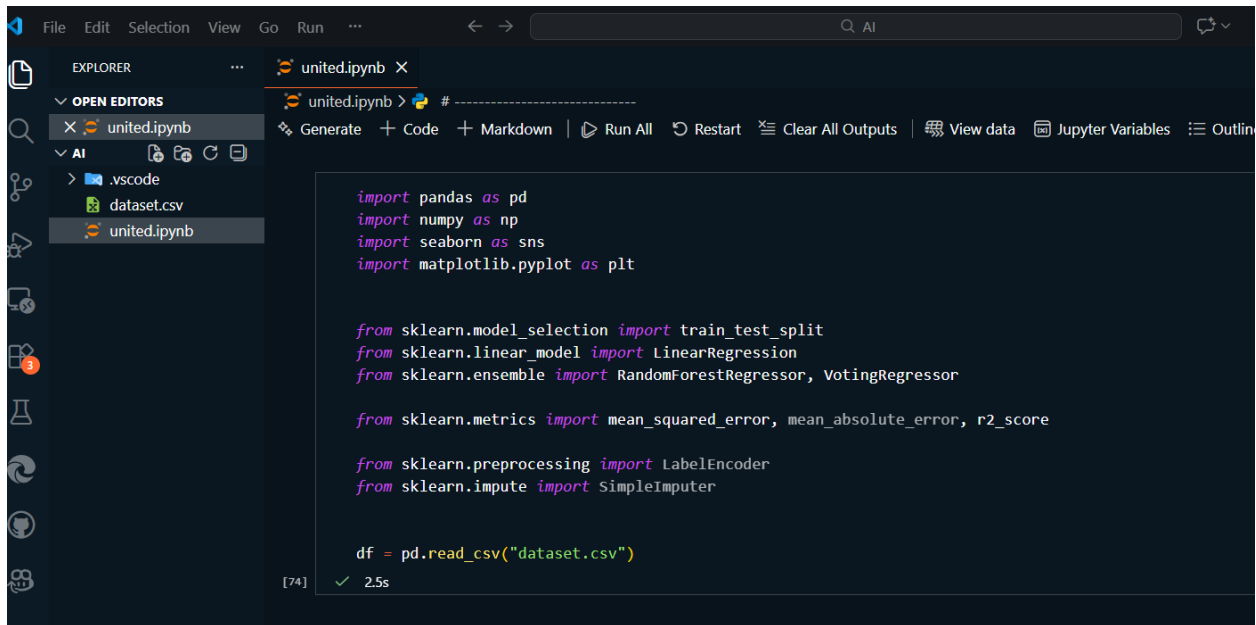
After cleaning the data, the cleaned dataset was viewed and verified by displaying columns, rows, and summary statistics. This helped confirm that all features were properly formatted and usable. Finally, the Linear Regression model was implemented. The dataset was split into training and testing sets, and Linear Regression was trained to predict Total Drug Spending based on selected features. This marks the initial implementation of machine learning in the project.

### 4.2. Tools, Technologies, and Libraries Used

- Python – Main programming language used for development.
- Pandas – Used for loading, cleaning, and manipulating the dataset.
- NumPy – Used for numerical operations.
- Scikit-learn – Used to implement Linear Regression, Random Forest, and Ensemble models.
- Matplotlib & Seaborn – Used for data visualization (histograms, heatmaps, line charts, bar chart.)
- Hugging Face – Source of the drug utilization dataset.
- VS Code – Used for project organization and script execution.

### 4.3. Explanation of Development

First, all the required libraries and dataset from hugging face imported.



```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, VotingRegressor

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

from sklearn.preprocessing import LabelEncoder
from sklearn.impute import SimpleImputer

df = pd.read_csv("dataset.csv")
```

[74] ✓ 2.5s

Figure 4: Importing dataset and libraries.

Checking Missing values



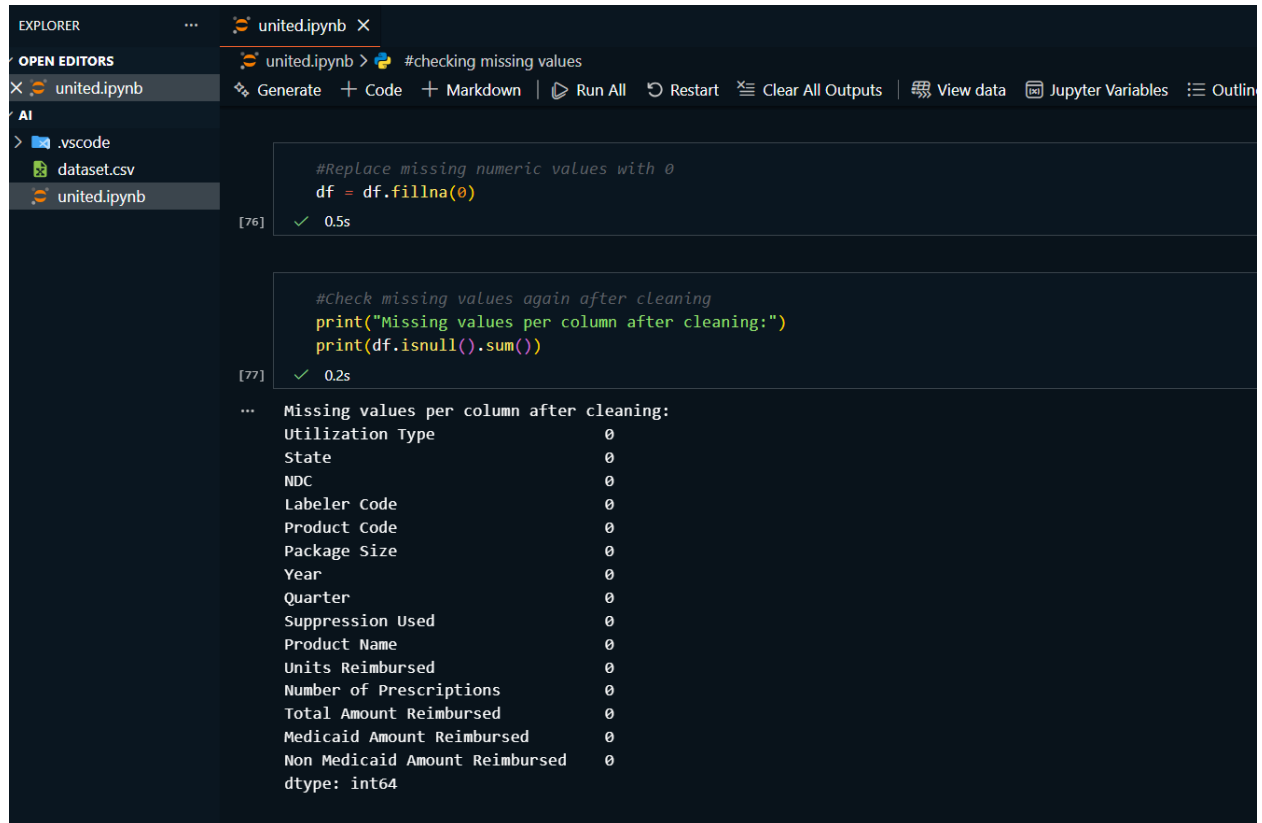
```
#checking missing values
print("Total rows:", len(df))
print("Missing values per column after cleaning:")
print(df.isnull().sum())
```

[75] ✓ 0.2s

```
Total rows: 2550443
Missing values per column after cleaning:
Utilization Type      0
State                  0
NDC                    0
Labeler Code          0
Product Code          0
Package Size          0
Year                  0
Quarter               0
Suppression Used      0
Product Name          0
Units Reimbursed      1283472
Number of Prescriptions 1283472
Total Amount Reimbursed 1283472
Medicaid Amount Reimbursed 1283472
Non Medicaid Amount Reimbursed 1283472
dtype: int64
```

Figure 5: Checking missing values

Checking missing values aging after cleaning and Replacing missing.



The screenshot shows a Jupyter Notebook interface with two code cells. The first cell contains a comment and a line of code to replace missing numeric values with 0. The second cell contains a comment and two lines of code to print the missing values per column after cleaning. The output of the second cell shows a table of missing values per column, all of which are 0.

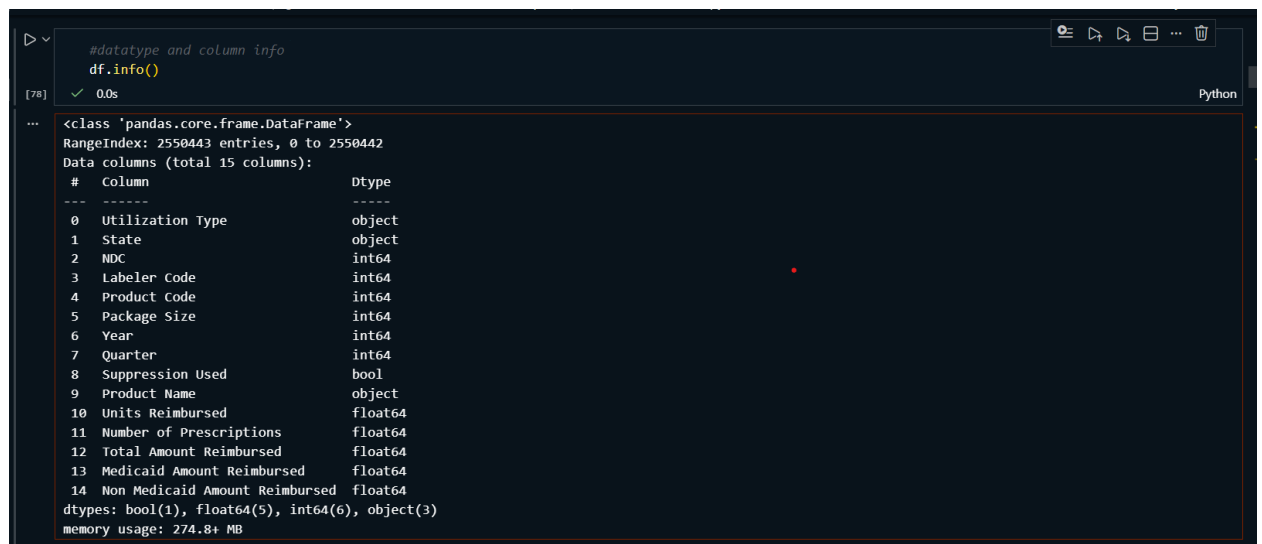
```
#Replace missing numeric values with 0
df = df.fillna(0)
```

```
#Check missing values again after cleaning
print("Missing values per column after cleaning:")
print(df.isnull().sum())
```

```
Missing values per column after cleaning:
Utilization Type      0
State                 0
NDC                   0
Labeler Code          0
Product Code          0
Package Size          0
Year                  0
Quarter               0
Suppression Used      0
Product Name          0
Units Reimbursed      0
Number of Prescriptions 0
Total Amount Reimbursed 0
Medicaid Amount Reimbursed 0
Non Medicaid Amount Reimbursed 0
dtype: int64
```

Figure 6: checking missing values aging after cleaning.

Checking datatype



The screenshot shows a Jupyter Notebook interface with a code cell containing a comment and a line of code to check the datatype of a DataFrame. The output shows the class of the DataFrame, the range index, the data columns, the dtypes of the columns, and the memory usage.

```
#datatype and column info
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2550443 entries, 0 to 2550442
Data columns (total 15 columns):
#   Column                Dtype
---  ---
0   Utilization Type      object
1   State                 object
2   NDC                   int64
3   Labeler Code          int64
4   Product Code          int64
5   Package Size          int64
6   Year                  int64
7   Quarter               int64
8   Suppression Used      bool
9   Product Name          object
10  Units Reimbursed      float64
11  Number of Prescriptions float64
12  Total Amount Reimbursed float64
13  Medicaid Amount Reimbursed float64
14  Non Medicaid Amount Reimbursed float64
dtypes: bool(1), float64(5), int64(6), object(3)
memory usage: 274.8+ MB
```

Figure 7: Checking datatype.

After statistical summary

|       | # NDC              | # Labeler Code     | # Product Code    | # Package Size     | # Year    | # Quarter |     |
|-------|--------------------|--------------------|-------------------|--------------------|-----------|-----------|-----|
| count | 2550443.0          | 2550443.0          | 2550443.0         | 2550443.0          | 2550443.0 | 2550443.0 |     |
| mean  | 40627092390.88394  | 40626.962592773096 | 1297.762200919605 | 21.890761330482587 |           | 2024.0    | 1.0 |
| std   | 28063301561.938324 | 28063.407964590955 | 2105.558269284561 | 27.52607676022795  |           | 0.0       | 0.4 |
| min   | 2010102.0          | 2.0                | 0.0               | 0.0                |           | 2024.0    |     |
| 25%   | 990798437.0        | 990.0              | 179.0             | 1.0                |           | 2024.0    |     |
| 50%   | 50742024001.0      | 50742.0            | 477.0             | 10.0               |           | 2024.0    |     |
| 75%   | 66689077508.0      | 66689.0            | 955.0             | 30.0               |           | 2024.0    |     |
| max   | 99999999920.0      | 99999.0            | 9999.0            | 99.0               |           | 2024.0    |     |

Figure 8: Screenshot of statistical summary.

Then show columns were loaded after displaying dataset

|   | Utilization Type | State | # NDC   | # Labeler Code | # Product Code | # Package Size |
|---|------------------|-------|---------|----------------|----------------|----------------|
| 0 | FFSU             | AK    | 2143380 | 2              | 1433           |                |
| 1 | FFSU             | AK    | 2143480 | 2              | 1434           |                |
| 2 | FFSU             | AK    | 2143611 | 2              | 1436           |                |
| 3 | FFSU             | AK    | 2144511 | 2              | 1445           |                |
| 4 | FFSU             | AK    | 2144527 | 2              | 1445           |                |
| 5 | FFSU             | AK    | 2145780 | 2              | 1457           |                |
| 6 | FFSU             | AK    | 2146080 | 2              | 1460           |                |
| 7 | FFSU             | AK    | 2147180 | 2              | 1471           |                |
| 8 | FFSU             | AK    | 2148480 | 2              | 1484           |                |
| 9 | FFSU             | AK    | 2149580 | 2              | 1495           |                |

Figure 9: Displaying dataset.

## Bar chart viewing top ten drugs by total spending

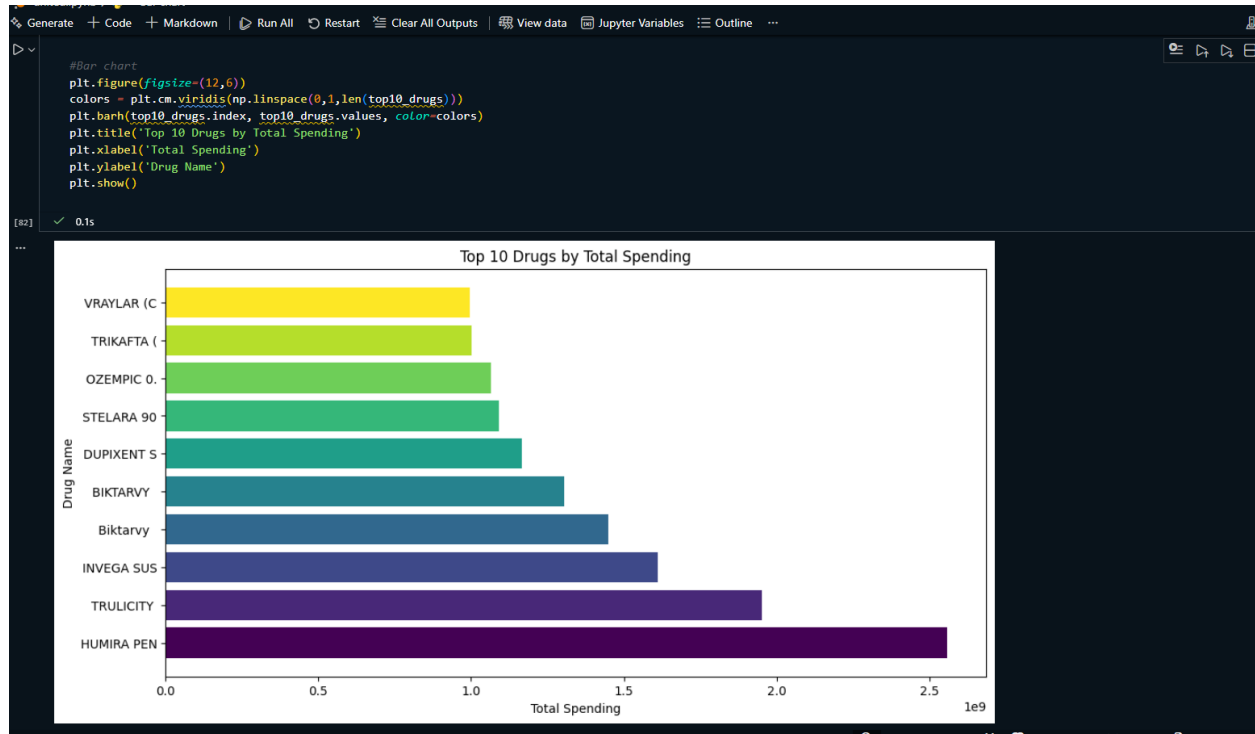


Figure 10:screenshot of bar chart.

First we remove rows where state is missing

```
# Remove rows where State is missing or XX
df_clean = df[df['State'] != 'XX']
```

Figure 11: Figure of removing state missing and X.

Then check result,

```
us_states = {state.abbr: state.name for state in us.states.STATES}

# Check result
print(us_states)
```

Output:

```
{'AL': 'Alabama', 'AK': 'Alaska', 'AZ': 'Arizona', 'AR': 'Arkansas', 'CA': 'California', 'CO': 'Colorado', 'CT': 'Connecticut', ...}
```

Figure 12: showing states.

Now, pie chart of spending by state

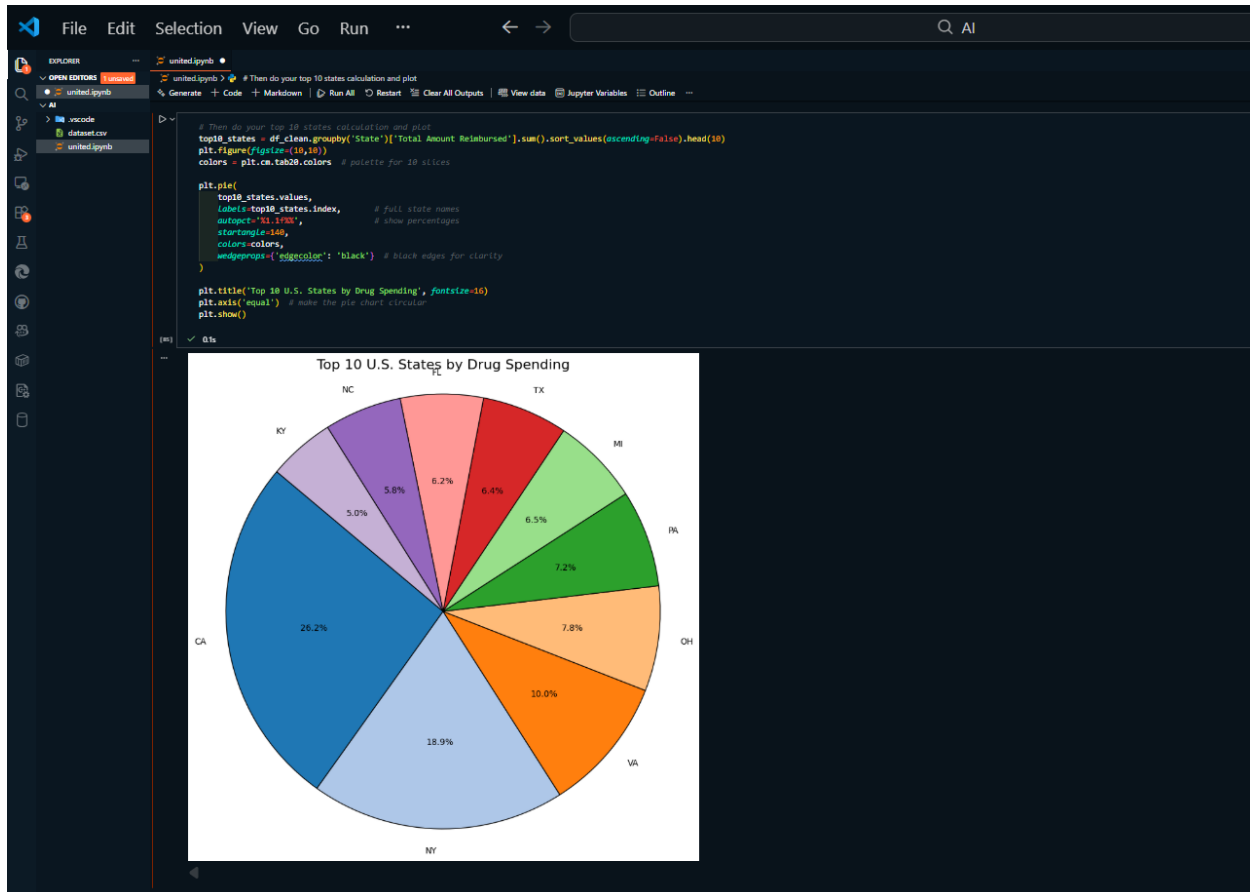


Figure 13: Top ten calculation by spending drugs by Bar chart.

Now line graph using year Quarter

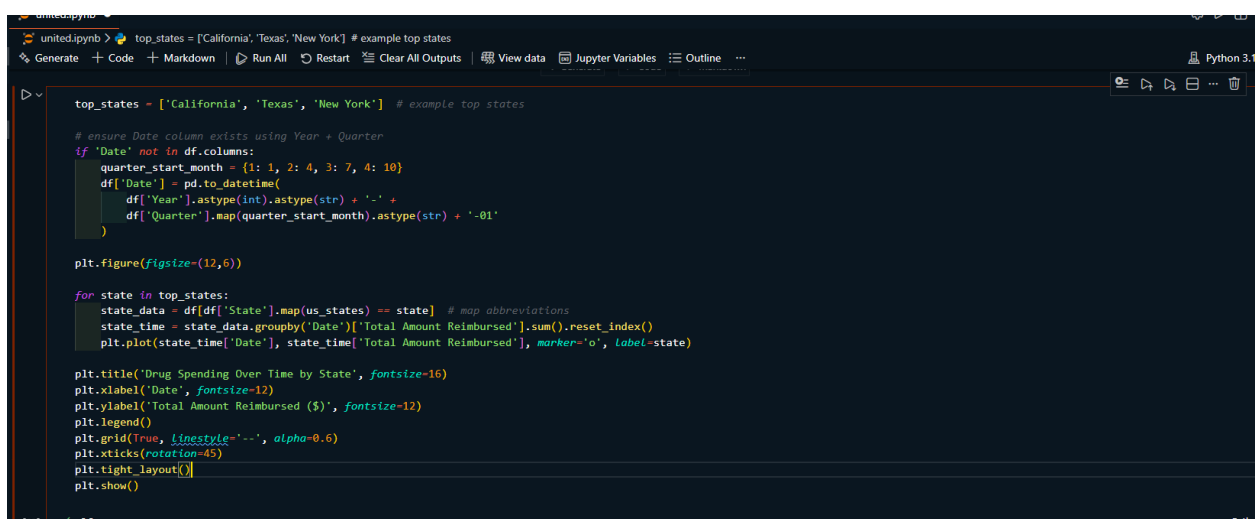


Figure 14: Screenshot of Pie chart

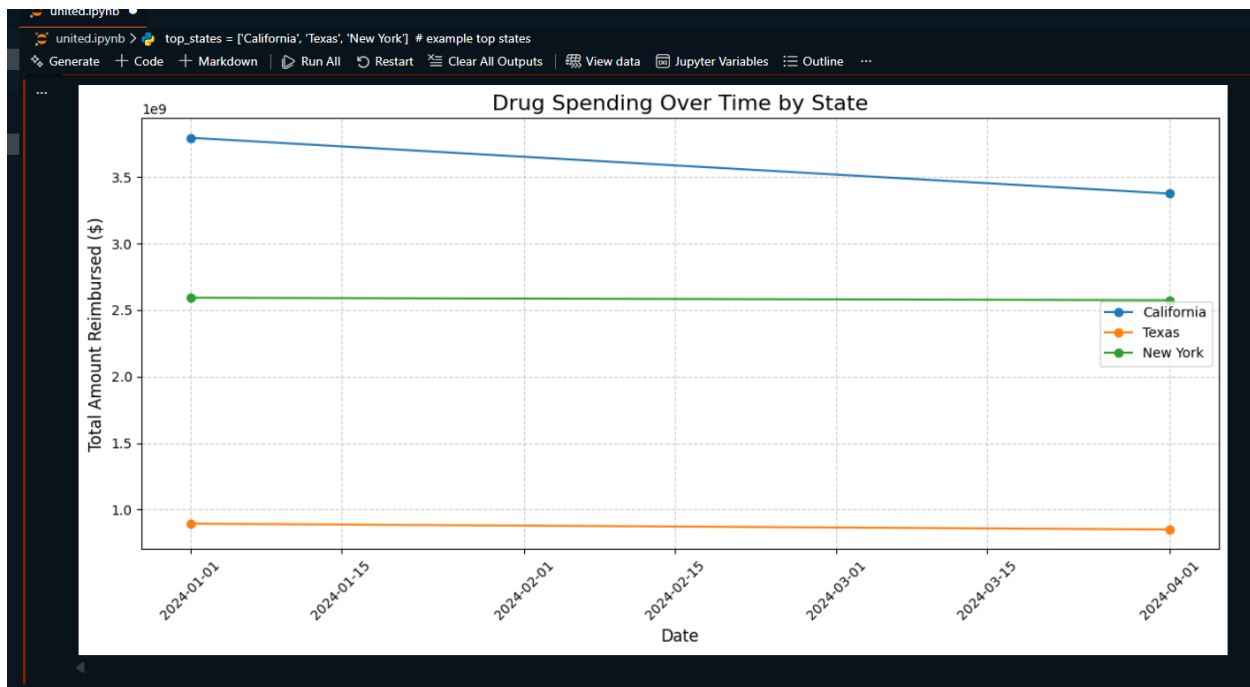


Figure 15: Drug spending over time by state

Now, Scale data to millions for better visualization



Figure 16: Correlation of heatmap of drug spending and prescription.

## 4.4. Findings

After all the processing are done now train to the model. The first model is trained with Linear regression.

### 4.4.1. Linear regression

```

# Linear regression model
# Feature(s)
X = df[['Number of Prescriptions']] # you can add more numeric features if needed

# Target
y = df['Total Amount Reimbursed']

# Split data: 80% training, 20% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

[182] ✓ 0.1s

# Create model
lr_model = LinearRegression()

# Train model
lr_model.fit(X_train, y_train)

[183] ✓ 0.0s

# Predict on test set
y_pred = lr_model.predict(X_test)

[184] ✓ 0.0s

# Mean Squared Error (MSE) and R² Score
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Linear Regression Model Performance:")
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"R² Score: {r2:.2f}")

[185] ✓ 0.0s

Linear Regression Model Performance:
Mean Squared Error (MSE): 1,109,496,524,506.93
R² Score: 0.06

```

Figure 17: screenshot of linear regression with model performance

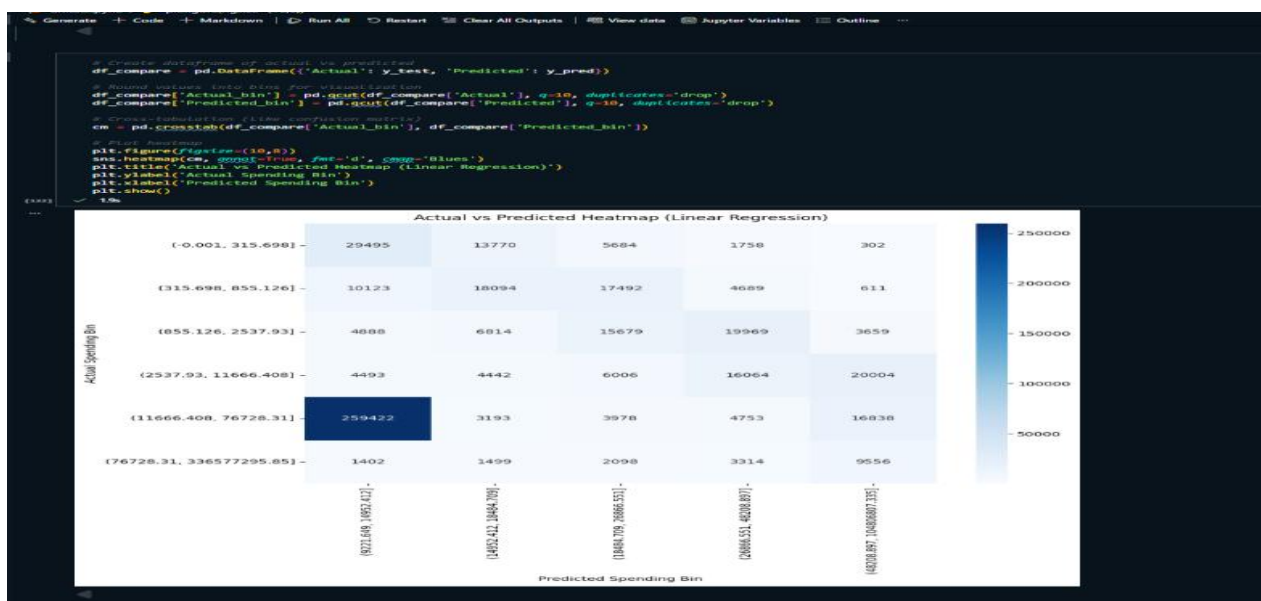


Figure 18: predicated heatmap in linear regression.

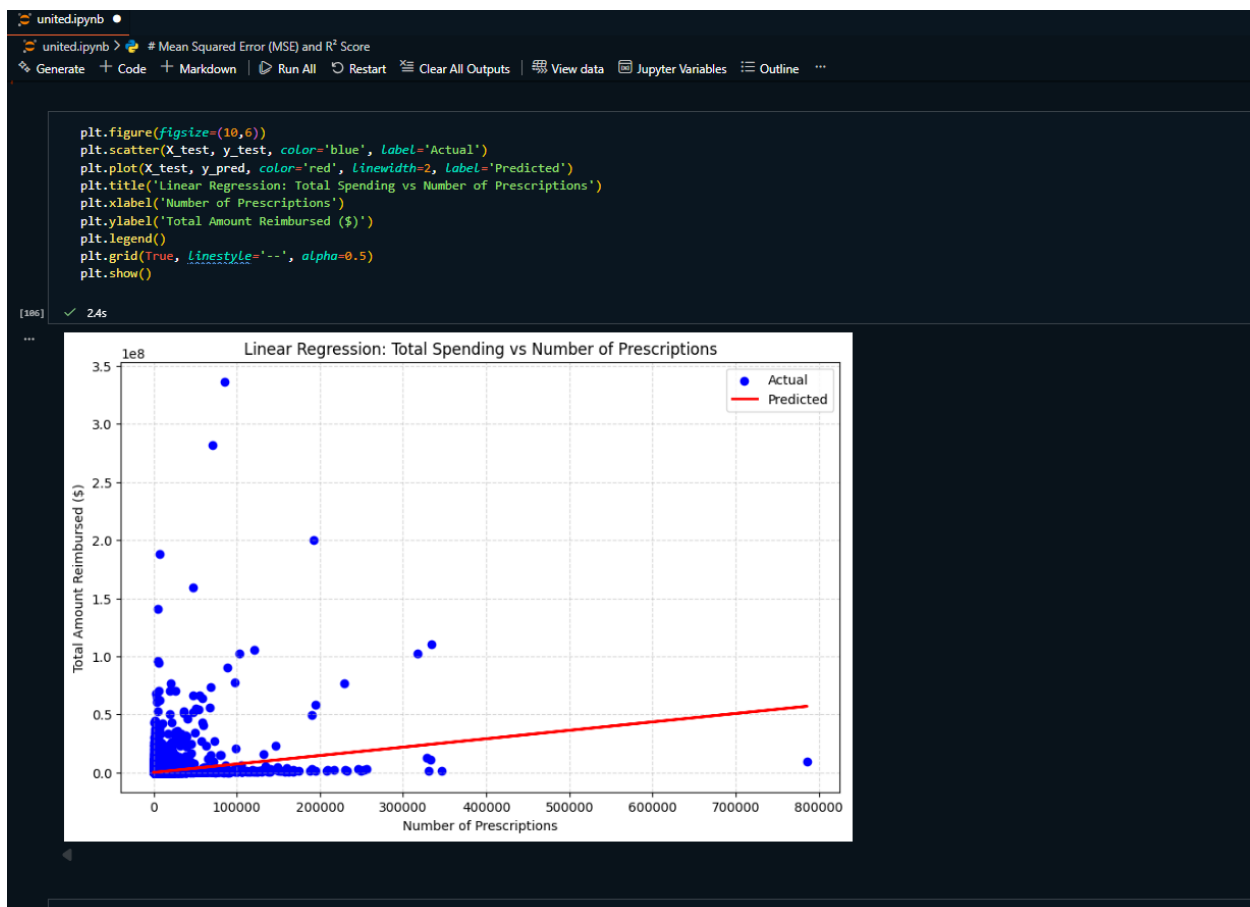


Figure 19: Total Spending and number of prescription.

### 4.4.2. Random Forest

```
#Random Forest Regressor model

# Features
X = df[['Number of Prescriptions']] # you can add more numeric columns

# Target
y = df['Total Amount Reimbursed']

# Split data: 80% train, 20% test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

✓ 0.2s

```
# Create the model
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)

# Train the model
rf_model.fit(X_train, y_train)
```

✓ 1m 48.3s

| RandomForestRegressor ⓘ ⓘ |                 |
|---------------------------|-----------------|
| ▼ Parameters              |                 |
| n_estimators              | 100             |
| criterion                 | 'squared_error' |
| max_depth                 | None            |
| min_samples_split         | 2               |
| min_samples_leaf          | 1               |
| min_weight_fraction_leaf  | 0.0             |
| max_features              | 1.0             |
| max_leaf_nodes            | None            |
| min_impurity_decrease     | 0.0             |
| bootstrap                 | True            |
| oob_score                 | False           |
| n_jobs                    | None            |
| random_state              | 42              |

Figure 20: Screenshot of Random Forest model.



The image shows a Jupyter Notebook interface with two cells. The first cell, labeled [109], contains Python code to calculate the Mean Squared Error (MSE) and the R-squared score for a Random Forest model. The code uses the `mean_squared_error` and `r2_score` functions from the `sklearn.metrics` module. The second cell, labeled [110], shows the output of the code, which prints the performance metrics.

```
[109] ✓ 3.1s Python
```

```
# Calculate Mean Squared Error and R2 Score
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Random Forest Model Performance:")
print(f"Mean Squared Error (MSE): {mse:,.2f}")
print(f"R2 Score: {r2:.2f}")
```

```
[110] ✓ 0.0s Python
```

```
''' Random Forest Model Performance:
Mean Squared Error (MSE): 1,729,060,319,426.10
R2 Score: -0.47
```

Figure 21: Random Forest model performance.



Figure 22: Random Forest of total spending and number of prescriptions.

#### 4.4.3. Ensemble Model

```
#Ensemble model with Voting Regressor

# Feature(s)
X = df[['Number of Prescriptions']] # add more numeric features if needed

# Target
y = df['Total Amount Reimbursed']

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

✓ 0.2s Python

Figure 23: Screenshot of ensemble model with voting Regressor.

```

# Individual models
lr_model = LinearRegression()
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)

# Ensemble model: Voting Regressor
ensemble_model = VotingRegressor(estimators=[('lr', lr_model), ('rf', rf_model)])

# Train the ensemble
ensemble_model.fit(X_train, y_train)

```

3] ✓ 1m 48.4s

**VotingRegressor** ⓘ ?

▼ Parameters

|            |                            |
|------------|----------------------------|
| estimators | [('lr', ...), ('rf', ...)] |
| weights    | None                       |
| n_jobs     | None                       |
| verbose    | False                      |

**lr**

**LinearRegression** ⓘ

▼ Parameters

|               |       |
|---------------|-------|
| fit_intercept | True  |
| copy_X        | True  |
| tol           | 1e-06 |
| n_jobs        | None  |
| positive      | False |

**rf**

**RandomForestRegressor** ⓘ

▼ Parameters

|                          |                 |
|--------------------------|-----------------|
| n_estimators             | 100             |
| criterion                | 'squared_error' |
| max_depth                | None            |
| min_samples_split        | 2               |
| min_samples_leaf         | 1               |
| min_weight_fraction_leaf | 0.0             |
| max_features             | 1.0             |
| max_leaf_nodes           | None            |
| min_impurity_decrease    | 0.0             |
| bootstrap                | True            |
| oob_score                | False           |
| n_jobs                   | None            |
| random_state             | 42              |

Figure 24: Train ensemble model with voting

```

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Ensemble Model Performance:")
print(f"Mean Squared Error (MSE): {mse:,.2f}")
print(f"R2 Score: {r2:,.2f}")

```

115] ✓ 0.0s

```

***
Ensemble Model Performance:
Mean Squared Error (MSE): 1,274,174,670,995.51
R2 Score: -0.08

```

Figure 25: Ensemble model performance.

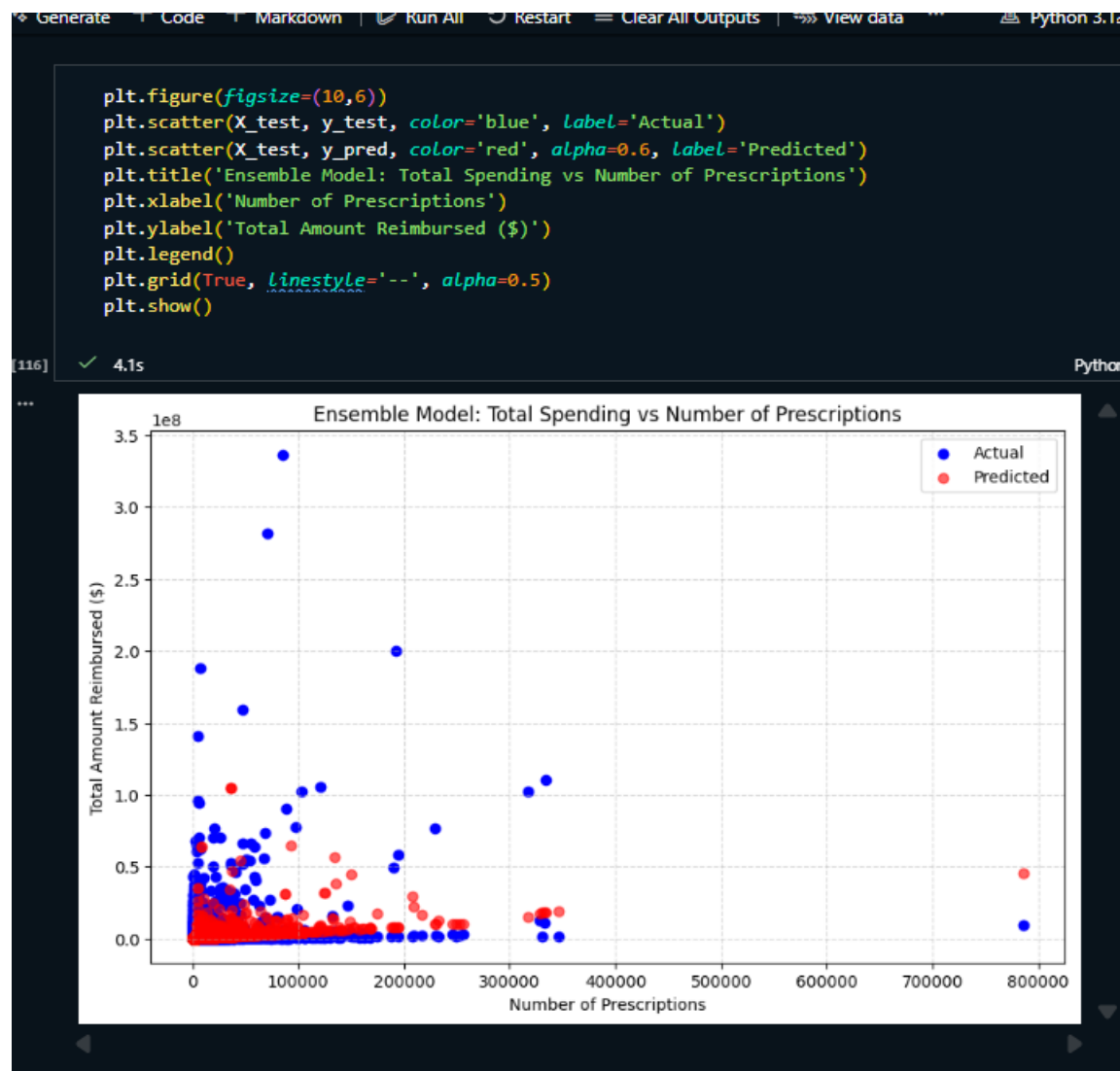


Figure 26: Total spending of ensemble model.

## 5. Conclusion

### 5.1. Analysis of the work done

In this project, we analyzed pharmaceutical drug spending using State Drug Utilization Data from the United States. First, the data was cleaned by handling missing values, and then different graphs were created to understand spending patterns, number of prescriptions, and state-wise drug usage. After data analysis, three machine learning models were used: Linear Regression, Random Forest, and an Ensemble Model. Linear Regression helped to understand the basic relationship between prescriptions and spending. Random Forest gave better results because it can handle complex data. The Ensemble Model performed the best by combining the strengths of different models.

### 5.2. Solution addresses real world problems

The solution helps to predict future drug spending, which is a major real-world problem for governments and hospitals. By knowing which states and drugs require more spending, health organizations can plan their budgets more effectively. This approach helps reduce waste, improve financial planning, and ensure better availability of medicines for patients.

### 5.3. Future work

In the future, more features such as time-based data, drug categories, and patient-related information can be added to improve accuracy. More advanced machine learning models can also be used for better prediction. Using updated and real-time data will make the system more reliable and useful for healthcare decision-making. The system can also be expanded to include more states and different types of medicines. Better data visualization dashboards can be created to help users understand the results easily. This model can also be used by hospitals and government offices to support long-term healthcare planning and cost control.

## 6. Bibliography

CMS.gov. (n.d.). *CMS Drug Spending*. Retrieved from cms.gov:  
<https://www.cms.gov/data-research/statistics-trends-and-reports/cms-drug-spending>

Datacatlog. (2025, December 12). *Data.gov*. Retrieved from State Drug Utilization Data 2024: <https://catalog.data.gov/dataset/state-drug-utilization-data-2024>

Medicaid.gov. (n.d.). *Medicaid.gov-Keeping America Healthy*. Retrieved from State Drug Utilization Data: <https://www.medicaid.gov/medicaid/prescription-drugs/state-drug-utilization-data?>

Utilization, M. S. (n.d.). *NBER*. Retrieved from National Bureau of Economic Research : <https://www.nber.org/research/data/medicaid-state-drug-utilization>